

Práctica 1

Punto 1

Inciso A

- Es un SO con características de Unix, pero es libre. Se distribuye bajo la licencia GPL, y su código fuente es público.
- Al ser software libre, Linux permite las siguientes libertades a sus usuarios:
 - Libertad de usar el programa con cualquier propósito.
 - Libertad de estudiar su funcionamiento.
 - Libertad para distribuir sus copias.
 - Libertad para mejorar los programas.
- Es *case-sensitive*.
- Es multiusuario
- Es multitarea y multiprocesador. Permite ejecutar varios programas al mismo tiempo, y es compatible con procesadores *multi-core*.
- Es altamente portable, por lo que se puede adaptar a distintos tipos de dispositivos.
- Puede usar distintos intérpretes de comandos o *Shells*. En particular, Debian usa *Bash*, pero esto es personalizable en Linux.
- Permite el manejo de usuarios y permisos.
- Trata todo como un archivo. Los dispositivos se tratan como archivos en el directorio */dev*.
- Sus directorios tienen distintos propósitos asignados.

Inciso B

- Características de *Windows* en comparación con Linux:
 - *Windows* es software propietario. Es decir que: no se distribuye su código fuente y no provee las 4 libertades de Linux.
 - No es *case-sensitive*.
 - Ambos son multiusuario, multitarea y multiprocesador. También puede administrar usuarios y permisos.
 - No es altamente portable. *Windows* solo se puede instalar en computadoras de uso general.
 - *Windows* también permite instalar otros intérpretes de comandos. Aunque por defecto usa *PowerShell* y *cmd*.
 - No trata a todo como un archivo. Los dispositivos se administran en un menú separado.
 - Sus directorios son de uso general.

Inciso C

- GNU es un sistema operativo similar a Unix, pero con la distinción de que es totalmente libre. Por eso está licenciado bajo la GPL.

Inciso D

- En 1983, el sistema GNU fue iniciado por Richard Stallman con el fin de crear un Unix libre.
- Con él se creó la licencia GPL.
- En 1985, R. Stallman creó la FSF (*Free Software Foundation*) para financiar el proyecto GNU.
- En 1990, GNU ya contaba con un editor de texto (*Emacs*), un compilador (*GCC*) y varias bibliotecas.
- Desde 1991, Linus Torvalds estaba trabajando en un kernel que sea software libre.
- En 1992, Torvalds y Stallman fusionaron ambos proyectos, y allí nació GNU/Linux. El kernel es, básicamente, el núcleo de un SO.

Inciso E

- La multitarea es el concepto en el que, administrando correctamente la memoria y el CPU, varios procesos pueden ejecutarse aparentemente de manera simultánea, aún cuando el CPU tiene un único *core*. Esto es mejorado ampliamente con procesadores multi-core.
- El tipo de multitarea que usa Linux se llama multitarea apropiativa (*preemptive multitasking*). El SO aquí tiene la capacidad de interrumpir un proceso para darle tiempo de CPU a otro.

Inciso F

- POSIX (*Portable Operating System Interface*) es una familia de estándares creados por el IEEE. Su objetivo es lograr la compatibilidad entre distintos SO.
- Permite que el código fuente de un software desarrollado para un SO que respeta el estándar POSIX, sea compilable en cualquier otro sistema POSIX.
- Muchos sistemas operativos actuales respetan este estándar, aunque algunos no en su totalidad. macOS lo respeta en su totalidad, mientras que Linux lo respeta parcialmente. En cambio, *Windows* no sigue a este estándar, aunque existen tecnologías que sirven para aumentar la compatibilidad de *Windows* con sistemas POSIX.

Punto 2

Inciso A

- Una distribución Linux o distribución GNU/Linux es un conjunto de aplicaciones reunidas que permiten brindar mejoras para instalar fácilmente un sistema operativo basado en GNU/Linux. Son “sabores” de GNU/Linux que, en general, se diferencian entre sí por las herramientas para configuración y sistemas de administración de paquetes de software para instalar. Las distribuciones son libres de usar cualquier tipo de *software*, ya sea libre o propietario. Algunas **distros** son:
 - **Ubuntu:** Es la distribución más popular, y tiene como objetivo facilitar la experiencia del usuario. Está basada en Debian.
 - **Mint:** Tiene una interfaz similar a *Windows*, para facilitar el cambio de dicho SO a Linux. Está basada en Ubuntu.
 - **Fedora:** Es la continuación del proyecto *Red Hat*. Su objetivo es estar a la vanguardia de los programas de *software* libre y de código abierto.
 - **Damn Small Linux:** Como su nombre lo indica, está diseñado para ser liviano y poder ejecutarse en *hardware* desactualizado.

Inciso B

- Como se mencionó, cada distribución tiene un foco particular (rendimiento en máquinas desactualizadas, facilidad de uso, etc) el kernel de Linux.

Inciso C

- Debian es la segunda distribución más popular de Linux. Tiene un énfasis en proveer estabilidad y *support* a largo plazo.
- Objetivo
 - Crear un sistema operativo libre, disponible para todo el mundo.
- Cronología
 - Debian comenzó a ser desarrollado por Ian Murdock en 1993.
 - Se consolidó como una de las distribuciones más populares a principios de los 2000.
 - En 2004, el comienzo de Ubuntu generó controversia en cuanto a la compatibilidad con sistemas derivados de Debian.
 - El sistema ha sufrido controversias en relación al uso de servicios no libres, pero continúa siendo una de las distribuciones más usadas hoy en día.

Punto 3

Inciso A

- Los componentes fundamentales de GNU/Linux son: el kernel, el *shell* y el *file system*.

Inciso B

- La estructura básica de GNU/Linux consiste en:
 - **Kernel:** es el núcleo del SO. Se encarga de administrar el uso del CPU y la memoria. De modo general, se encarga de que el *software* y el *hardware* puedan trabajar juntos.

- **Las aplicaciones base:** incluyen herramientas básicas como compiladores y librerías, así como programas que usa el usuario.
- **Shell:** es el intérprete de comandos. Dichos comandos pueden ser literales, en un shell con interfaz de consola (CLI) o pueden ser acciones del usuario en una interfaz gráfica (GUI).

Punto 4

Inciso A

- Funciones principales del kernel:
 - Ejecutar programas y gestionar dispositivos.
 - Coordinar el *software* y el *hardware*
 - Administra la memoria, el CPU y la E/S.
 - Es un núcleo monolítico e híbrido
 - * Es monolítico porque es un solo proceso. Si se cuelga algo en el kernel, se cuelga todo.
 - * Es híbrido porque se le pueden cargar y descargar *drivers*, que le agregan funcionalidades distintas.
 - * Todo el código del kernel se ejecuta en modo privilegiado.

Inciso B

- Sí, es posible tener más de un kernel de GNU/Linux en la misma máquina. De la misma manera que el dual booting se puede usar para tener Linux y Windows en la misma PC, también podría usarse para tener Linux y una versión más vieja de Linux. En general, cuando el kernel se actualiza, no borra los archivos del kernel viejo, sino que ambos coexisten en el directorio `/boot/`.

Inciso

- Se encuentran en el directorio `/boot/`, y comienzan con la secuencia “`vmlinuz-`”. Se puede ver todas las versiones de kernel que hay en una máquina Linux usando este comando en la terminal: `ls -lh /boot/vmlinuz-*`

Punto 5

Inciso A

- El *shell*, consola o intérprete de comandos, es un programa que actúa como interfaz para comunicar al usuario con el sistema operativo. En una interfaz CLI, el usuario escribe un comando, el *shell* lo interpreta, y se lo entrega al SO para su ejecución.

Inciso B

- Algunos *shells* de GNU/Linux son:
 - **Bourne Shell:** Ubicado en `/bin/sh`, está disponible en todas las versiones de UNIX y es lo suficientemente básico para que funcione en todas las plataformas.
 - **Bourne Again Shell o BASH:** Ubicado en `/bin/bash`, es uno de los *shells* más potentes y avanzados. Tiene licencia GNU. Algunas de sus funciones son:
 - * Historial de comandos ejecutados, conservado entre sesiones.
 - * Auto completado de nombres de comandos presionando TAB.
 - **Z Shell o Zsh:** Ubicado en `/bin/zsh`, está diseñado para su uso interactivo. Tiene incorporadas características de *Bash*, y es el *default* en macOS ### Inciso C
- Los comandos propios de una *shell* se encuentran en su directorio particular. Por ejemplo, los comandos propios de *Bash* se encuentran en `/bin/bash`. En cambio, los comandos externos pueden estar en `/bin`, `/usr/bin`, `/usr/local/bin` o cualquier otra ubicación si se la agrega a la variable PATH. Se pueden ver todas las ubicaciones de la variable PATH con el siguiente comando:

```
echo $PATH
```

Inciso D

- El *shell* no forma parte del kernel de GNU/Linux porque, justamente, sirve para que el usuario pueda enviarle órdenes al mismo sin interactuar directamente con él. Si el *shell* fuese parte del kernel, esto sería imposible.

Inciso E

- Sí, cada usuario puede usar un intérprete de comandos diferente según su elección. En Linux, se puede ver el *shell* asignado a cada usuario en el archivo `/etc/passwd`. Para buscar el de un usuario, se puede usar el siguiente comando:

```
grep "nombre_usuario" /etc/passwd | cut -d: -f7
```

Punto 6

Inciso A

- Un sistema de archivos o *file system* es la manera en la que, en una computadora, se administran y organizan los archivos. Esto incluye:
 - **Métodos de acceso:** cómo se acceden los datos contenidos en el archivo.
 - **Manejo de archivos:** cómo actúan los mecanismos para almacenar, referenciar, compartir y proteger los archivos.
 - **Manejo de la memoria secundaria:** Cómo se administra el espacio para los archivos en memoria secundaria.
 - **Mecanismos de integridad:** con qué métodos se garantiza la incorruptibilidad del archivo.

Inciso B

- El contenido de cada uno de los directorios principales de Linux lo determina el FHS (*Filesystem Hierarchy Standard*). Estos son:
 - `/home` son los archivos personales del usuario.
 - `/dev` son los dispositivos conectados.
 - `/var` son archivos que cambian con frecuencia. En `/var/log` se guardan los *logs* de los programas.
 - `/etc` son las configuraciones de las bases de datos.
 - `/bin` son archivos binarios y ejecutables.
 - `/sbin` se usa para almacenar programas esenciales del sistema, que usará el administrador del mismo.
 - `/usr` son subdirectorios con archivos de programas y configuración del sistema.
 - `/boot` son los archivos necesarios para el arranque de la máquina.
 - `/root` es el directorio home del superusuario root.
 - `/tmp` contiene archivos no persistentes que generan los programas.
 - `/proc` contiene ficheros que hacen referencia a procesos que corren en el sistema, y le permiten obtener información acerca de qué programas y procesos están corriendo en un momento dado.
 - `/lib` contiene código de librerías que utilizan muchos programas.

Inciso C

- Algunos sistemas de archivos soportados por GNU/Linux son: ext3, ext4, ReiserFS y XFS

Punto 7

Inciso A

- Una partición de disco es una región de almacenamiento secundario, que se crea con el fin de que dicha área pueda manejarse de manera independiente a las otras.
- Tipos de particiones:
 - **Partición primaria:** división cruda del disco (solo puede haber 4 por disco). Se almacena información de la misma en el MBR.
 - **Partición extendida o secundaria:** sirve para contener particiones lógicas en su interior. Solo puede existir una partición de este tipo por disco. No se define un tipo de *file system* directamente sobre ella. Es decir, no se guardan archivos en la partición extendida directamente, sino que dentro de la partición extendida, se crea una partición lógica. Es en esa partición lógica donde se guardan los archivos.
 - **Partición lógica:** ocupa la totalidad o parte de la partición extendida y se le define un tipo de *file system*. Las particiones de este tipo se conectan como una lista enlazada entre sí.

Inciso B

- IDE era la interfaz estándar para la identificación de HDDs hasta que fue reemplazada por SATA. Los nombres de sus discos y particiones eran así:
 - `/dev/hda`: configurado como Master en el 1er bus IDE
 - `/dev/hdb`: configurado como Slave en el 1er bus IDE
 - `/dev/hdc`: configurado como Master en el 2do bus IDE
 - `/dev/hdd`: configurado como Slave en el 2do bus IDE
 - Las particiones primarias se numeran del 1 al 4, con las lógicas usando los números del 5 en adelante.
- SATA es la interfaz estándar usada actualmente por usuarios comunes. En cambio, la forma evolucionada de SCSI es el estándar para servidores. Sin embargo, la nomenclatura de ambos es idéntica en Linux:
 - `/dev/sda` disco físico 1
 - `/dev/sdb` disco físico 2
 - `/dev/sdc` disco físico 3, etc...
 - Las particiones primarias se numeran del 1 al 4, con las lógicas usando los números del 5 en adelante. Solo las particiones primarias pueden marcarse como *bootables*.

Inciso C

- Se necesita 1 sola partición primaria, que aloja al directorio raíz (`/`).
- Las identificaciones y puntos de montaje serían:
 - Disco físico: `sda`
 - * Partición `/`:
 - ID: `sda1`
 - Punto de montaje: `/ ###` Inciso D
- DUDA distintos esquemas de particionamiento para distintos usos

Inciso E

- Sí, Linux puede leer las particiones que usan *file systems* propios de Windows como FAT o NTFS. Sin embargo, lo opuesto no es verdad, ya que *Windows* no puede leer particiones `ext4`.

Inciso F

- Existen particionadores destructivos y no destructivos.
 - **Particionadores destructivos**: solo permiten crear o eliminar particiones. Ej: **Fdisk**
 - **Particionadores no destructivos**: permiten crear, eliminar, fusionar, dividir o editar particiones. Ej: **Gparted**, administrador de discos de *Windows*.

Punto 8

Inciso A

- La BIOS (*Basic Input/Output System*) es *firmware* que se almacena en un *chip* directamente soldado en la placa madre. Contiene la información necesaria para arrancar una computadora, con su última acción siendo leer el MBC del MBR, que comienza la carga del SO.

Inciso B

- UEFI (*Unified Extensible Firmware Interface*) realiza las mismas tareas que la BIOS, y es su reemplazo en computadoras modernas. A diferencia de la BIOS, que era propietario de IBM, UEFI es un estándar abierto de la industria. En vez de ejecutar ciegamente los primeros 512 bytes del disco, tiene en su placa madre un puntero a la tabla de particiones GPT. De aquí sabe a dónde ir para llegar a la partición especial ESP, que es un file system FAT32, que UEFI puede leer. En dicha partición, busca el archivo `.efi` de arranque y lo ejecuta.

Inciso C

- El MBR (*Master Boot Record*) son 512 bytes de un disco, que tienen información especial, y están siempre ubicados en el cilindro 0, cabeza 0, sector 1. Los primeros 446 bytes tienen instrucciones rudimentarias para que arranque el sistema operativo. Luego tiene la tabla de particiones primarias, que ocupa 64 bytes. Finalmente, se firma con 2 bytes especiales. La limitación de tener solo 4 particiones primarias viene de que la información de cada partición ocupa 16 bytes, y la tabla de particiones tiene un tamaño de 64 bytes.

Inciso D

- Las siglas GPT significan *GUID Partition Table* y son el reemplazo de la tabla de particiones del MBR, eliminando la limitación de 4 particiones primarias. De cada partición se almacena el nombre y las coordenadas de inicio y fin.

Inciso E

- Una vez que la BIOS o UEFI terminó su ejecución, no se cargó el sistema operativo en sí. Lo que se cargó fue un gestor de arranque, que es un programa con más complejidad, con instrucciones para iniciar un SO. Con el estándar UEFI, se almacenan en una partición especial llamada ESP (*EFI System Partition*). En cambio, con el estándar BIOS, para inicializar el gestor de arranque, la información de su ejecución tiene que estar en el primer sector de la partición marcada como *bootable*. Algunos gestores de arranque son:
 - **GRUB:** es el estándar para Linux.
 - **Windows Boot Manager:** es el estándar en *Windows*.

Inciso F

DUDA preguntar esto porque es importante

- Con estándar BIOS:
 - Se prende la computadora.
 - Se ejecuta el código ubicado en el cilindro 0, cabeza 0, sector 1 del disco. Aquí está el MBR.
 - Primero se ejecuta el MBR, con instrucciones simples que leen la tabla de particiones.
 - Se lee la tabla de particiones, y se busca aquella con el flag de *bootable* activo.
 - Una vez que se llega a la partición *bootable*, se ejecuta el primer sector de la misma, que tiene los primeros pasos de ejecución para inicializar el gestor de arranque.
 - Una vez ejecutado el gestor de arranque, este permite cargar el sistema operativo.
- Con estándar UEFI:
 - Se prende la computadora
 - En la placa madre del chip *UEFI*, se tiene almacenada la ubicación del archivo del gestor de arranque.
 - En el principio del disco, se lee la tabla de particiones GPT, para poder ubicar la partición con ID de UEFI (ESP).
 - * **Nota:** En realidad, en el primer sector del disco sigue estando el viejo MBR, que se mantiene por cuestiones de retro-compatibilidad. Es en el siguiente sector a ese donde se almacena la tabla GPT.
 - En la partición ESP, se busca el archivo del gestor de arranque (la ubicación del mismo estaba almacenada en el chip UEFI, así que sabe donde buscarlo)
 - Se revisa que no se trate de un archivo malicioso.
 - Se ejecuta el gestor de arranque.
 - El gestor de arranque carga el SO.

Inciso G

DUDA es lo mismo que cualquier otro SO (el arranque en Linux)?

Inciso H

- DUDA que pasos se ejecutan al apagar una maquina Linux

Inciso I

- Sí, es posible, ya que hay gestores de arranque como GRUB que muestran un menú al ejecutarse. En este menú se encuentran los distintos archivos de arranque de cada SO instalado en una PC.

Punto 9

Inciso A

- DUDA a que se refiere como se identifica a un archivo en Linux

Inciso B

- **Vim:** es un editor de texto que se puede usar tanto por consola como con una interfaz gráfica (modo GUI). Tiene 7 modos, aunque en la cátedra se nombraron 3:
 - Modo *insert* (activado con la tecla I): es para editar contenido como en un editor moderno.
 - Modo visual (activado con la tecla V): es para seleccionar áreas de texto. Se pueden correr comandos sobre las áreas seleccionadas.
 - Modo normal (activado con la tecla Esc): es para comandos de editor. Es el modo por *default*.
 - Algunos de sus comandos son:
 - * w: escribir cambios
 - * q o q!: salir del editor
 - * dd: cortar
 - * y: copiar al portapapeles
 - * p: pegar desde el portapapeles
 - * u: deshacer
 - * /frase: busca “frase” dentro del archivo
- **Nano:** es un editor de texto que usa una interfaz por consola. Es similar al editor Pico, y fue creado para que haya una alternativa libre a dicho editor de Unix (fue una historia parecida a GNU con el propio Unix). El desarrollador principal del proyecto lo abandonó en 2016 por problemas con la *Free Software Foundation*.
- **Mcedit:** es un editor de texto que viene incluido en el programa de manejo de archivos conocido como *Midnight Commander*. Se puede ejecutar como un programa aparte (fuera de *Midnight Commander*) y tiene características como: resaltado de sintaxis en muchos lenguajes, macros, indentación automática, uso de *mouse*, portapapeles, entre otras.

Inciso C

- Se debe realizar esta serie de comandos y acciones, teniendo Vim instalado:

```
cd ~
```

```
vim prueba.exe
```

Entrar a modo **insert** apretando la tecla I

Escribir nombre y número de alumno, como si fuese un .txt normal (se puede usar el Enter para saltar de lí

Apretar Esc para entrar a modo normal. Guardar el archivo con el siguiente comando:

```
:wq
```

- Para comprobar que se creó el archivo, ejecutar el siguiente comando:

```
ls -l prueba.exe
```

Inciso D

- cat: su nombre viene de *concatenate*. Permite mostrar archivos de texto en la terminal con distintos formatos. Algunas de sus funciones son:
 - Ver un archivo: `cat archivo.txt` muestra todo el texto en la pantalla.
 - Numerar líneas: `cat -n archivo.txt` muestra el texto con números al lado de cada línea.
 - Unir archivos: `cat archivo1.txt archivo2.txt > nuevo.txt` junta dos archivos en uno nuevo.

- Crear un archivo: `cat > archivo.txt` te deja escribir texto nuevo y guardarlo al pulsar Ctrl + D.
- `more`: sirve para ver el contenido de un archivo de texto largo pantalla por pantalla, para de esta manera no saturar la terminal.
 - Sus controles son:
 - * Barra espaciadora: Avanza una página completa.
 - * Tecla Enter (Intro): Avanza una sola línea.
 - * Tecla Q: Sale del comando y vuelve a la terminal
 - `more archivo.txt` sería el comando
 - Se puede usar para leer la salida de otros comandos de la siguiente manera:
 - * `ls | more` mostraría página por página todos los archivos y directorios de la carpeta en la que estás parado.

Inciso E

- El comando `file` sirve para identificar el tipo de archivo con el que se trata. Un ejemplo de uso sería:

`file prueba.exe`

- Internamente, hace 3 pasos:
 - Realiza una *system call* llamada `stat()`, lo que le permite leer algunos metadatos de un archivo. Entre ellos, está la información de si el archivo es un archivo regular, un dispositivo o un directorio. Si es un directorio, lo reporta.
 - Lee los “números mágicos” del archivo. Estos suelen ser los primeros bytes del mismo, y son una firma digital. El comando luego los compara con una base de datos interna, que asocia a cada *magic number* con un tipo de archivo.
 - Si no tiene ningún número mágico reconocible, lee los primeros bloques (generalmente unos pocos kb), buscando palabras clave (ej: *program* para Pascal) o una codificación específica como ASCII o UTF-8 para determinar el tipo de archivo.

Inciso F

- **Nota:** se recomienda visualizar esta tabla en una pantalla ancha. Si aparece formateada de manera errónea, aquí hay un [link a la tabla en pdf](#).

Nombre	Significado en inglés	Comportamiento
<code>cd</code>	<i>change directory</i>	Sirve para entrar a un directorio. Ej: <code>cd documents</code>
<code>mkdir</code>	<i>make directory</i>	Crea un directorio nuevo. Ej: <code>mkdir carpeta_a_crear</code>
<code>rmdir</code>	<i>remove directory</i>	Elimina un directorio. Ej: <code>rmdir carpeta_a_borrar</code>
<code>ln</code>	<i>link</i>	Crea un enlace a un directorio o archivo (como un acceso directo) en la carpeta donde estás ubicado. Ej: <code>ln -s [archivo_original] [nombre_del_enlace]</code> . En este caso, <code>-s</code> indica que es un enlace simbólico (si se borra o mueve el archivo, deja de funcionar)
<code>tail</code>	<i>tail</i>	Muestra las últimas líneas de un archivo de texto. Ej: <code>tail -n 20 archivo.txt</code> <code>-n 20</code> hace que muestre 20 líneas
<code>locate</code>	<i>locate</i>	Busca en una base de datos interna un archivo Ej: <code>locate nombre_archivo</code> . Parámetros: <code>-i</code> ignora mayúsculas y minúsculas, <code>-n 5</code> limita el número de resultados a 5.

Nombre	Significado en inglés	Comportamiento
ls	<i>list</i>	Muestra una lista de todos los archivos y directorios de mi ubicación. Parámetros: -l incluye permisos, el dueño, el tamaño y la fecha, -a incluye los ocultos que empiezan con (.), -lh muestra el tamaño en unidades legibles
pwd	<i>print working directory</i>	Muestra la ruta absoluta del directorio actual. Parámetros: -P ignora enlaces simbólicos (los que crea ln), -L tiene en cuenta enlaces simbólicos.
cp	<i>copy</i>	Copia un archivo con otro nombre en el mismo lugar, o con el mismo nombre en otro lugar. Ej: <code>cp archivo.txt nuevo_nombre.txt</code> o <code>cp archivo.txt /nueva/ubicacion/</code> . Parámetros: -i pregunta antes de sobrescribir, -v muestra el paso a paso.
mv	<i>move</i>	Sirve para mover o renombrar archivos y directorios. Ejemplo para mover: <code>mv archivo.txt /home/usuario/documentos/</code> . Ejemplo para renombrar: <code>mv viejo.txt nuevo.txt</code>
find	<i>find</i>	Sirve para buscar un archivo por distintos criterios (nombre, tamaño, tipo, fecha, etc). Ejemplo: <code>find . -size 54k</code> busca archivos de 54 kilobytes.

Punto 10

Inciso A

- `mkdir ISOCS0`

Inciso B

- `cd ISOCS0`

Inciso C

- `touch isocso.txt`
- `touch isocso.csv`

Inciso D

- `ls`

Inciso E

- `pwd`

Inciso F

- `find . -iname "iso\*"`

Inciso G

- `df -h`

Inciso H

- `w`

Inciso I

- `echo "Franciso Barberena" > isocso.txt`

Inciso J

- `tail isocso.txt`

Punto 11

- `man`
 - Sirve para mostrar la documentación oficial o página de manual de cualquier otro comando del sistema.
 - `/usr/bin/man`
 - Parámetros:
 - * `-whatis`
 - * `-k` [palabra clave]. Busca comandos que tengan la palabra clave en su descripción.
- `shutdown`
 - Se utiliza para apagar, reiniciar o programar el apagado del sistema de forma segura.
 - `/usr/sbin/shutdown`
 - Parámetros
 - * `-h` / `-halt`: Apaga el equipo deteniendo todos los procesos.
 - * `-P` / `-poweroff`: Apaga el equipo y corta la energía (acción por defecto).
 - * `-r` / `-reboot`: Reinicia el sistema.
 - * `-c` / `-cancel`: Cancela un apagado o reinicio programado.
 - * `-k` / `-kmsg`: Envía el mensaje de advertencia a los usuarios pero no apaga el sistema.
- `reboot`
 - Se utiliza para reiniciar el SO.
 - `/usr/sbin/reboot`
 - Parámetros
 - * `-f` (`-force`) Fuerza el reinicio inmediato. No contacta al gestor de inicio `init` ni a `systemd`. Puede causar pérdida de datos no guardados.
 - * `-p` (`-poweroff`) Apaga el equipo por completo en lugar de reiniciarlo. Actúa como el comando `poweroff`.
 - * `-w` (`-wtmp-only`) No reinicia el sistema. Solo registra la acción de reinicio en el archivo de historial `/var/log/wtmp`.
 - * `-d` (`-no-wtmp`) Reinicia el sistema pero suprime la escritura del registro en el archivo `/var/log/wtmp`.
 - * `-halt` Detiene el sistema (pasa a estado de suspensión/halt) en lugar de apagarlo o reiniciarlo.
- `halt`
 - Se utiliza para apagar el equipo de forma inmediata, rompiendo el flujo normal del sistema y deteniendo la CPU.
 - `/usr/sbin/halt`
 - Parámetros
 - * `-p` / `-poweroff`: Apaga el equipo y corta la corriente eléctrica de la máquina.
 - * `-reboot`: Reinicia el sistema en lugar de solo detenerlo (funciona igual que el comando `reboot`).
 - * `-f` / `-force`: Fuerza la parada inmediata. No contacta con el gestor de servicios (`systemd`). Puede causar pérdida de datos si hay archivos abiertos.

- * -w / -wtmp-only: No apaga el sistema. Solo registra el evento de parada en el archivo de historial /var/log/wtmp.
- * -d / -no-wtmp: Apaga el sistema pero no registra el evento en el archivo /var/log/wtmp.
- * -no-wall: Apaga el equipo sin enviar el mensaje de advertencia a los usuarios conectados a la terminal.
- **uname**
 - Sirve para mostrar información detallada del sistema operativo y del kernel.
 - /usr/bin/uname
 - Parámetros
 - * -a o -all: Muestra toda la información del sistema en orden fijo.
 - * -s o -kernel-name: Muestra el nombre del núcleo (ej. Linux).
 - * -n o -nodename: Muestra el nombre de host o red del equipo.
 - * -r o -kernel-release: Muestra la versión o subtipo del kernel.
 - * -v o -kernel-version: Muestra la fecha y detalles de compilación del kernel.
 - * -m o -machine: Muestra la arquitectura del hardware de la máquina (ej. x86_64).
 - * -p o -processor: Muestra el tipo de procesador.
 - * -i o -hardware-platform: Muestra la plataforma de hardware.
 - * -o o -operating-system: Muestra el nombre del sistema operativo (ej. GNU/Linux)
- **dmesg**
 - Se utiliza para examinar y controlar el búfer de anillos del kernel. Muestra los mensajes generados por el núcleo del sistema, especialmente durante el arranque y al conectar hardware.
 - /usr/bin/dmesg
 - Parámetros
 - * -T (o -ctime): Muestra marcas de tiempo legibles por humanos (fecha y hora local) en lugar del tiempo en segundos desde el arranque.
 - * -C (o -clear): Limpia por completo el búfer de mensajes del kernel.
 - * -c (o -read-clear): Muestra el contenido actual del búfer y luego lo limpia.
 - * -w (o -follow): Modo interactivo. Espera y muestra nuevos mensajes en tiempo real (ideal para ver qué pasa al conectar un USB).
 - * -L (o -color): Organiza la salida usando colores para facilitar la lectura de advertencias y errores.
 - * -l (o -level): Filtra la salida por niveles de criticidad (separados por comas). Niveles: emerg, alert, crit, err, warn, notice, info, debug.
 - * -f (o -facility): Filtra por el componente que generó el mensaje. Componentes: kern, user, mail, daemon, auth, syslog, lpr, news.
 - * -n (o -console-level): Configura el nivel de los mensajes que se enviarán directamente a la consola de texto.
 - * -s (o -buffer-size): Define el tamaño del búfer utilizado para consultar al kernel (por defecto suele ser 16392 bytes).
 - * -H (o -human): Activa una salida amigable: habilita paginación automática, colores y marcas de tiempo amigables
- **lspci**
 - Sirve para mostrar información sobre todos los periféricos conectados al sistema, como tarjetas gráficas, de red, de sonido y controladores de disco.
 - /usr/bin/lspci
 - -v, -vv y -vvv aumentan el nivel del detalle de la información.
- **at**
 - Sirve para programar la ejecución de tareas una sola vez en un momento determinado.
 - /usr/bin/at
 - Parámetros
 - * -c: Muestra el contenido de un trabajo programado antes de que se ejecute.
 - * -f: Lee los comandos desde un archivo en lugar de la entrada estándar.
 - * -m: Envía un correo electrónico al usuario cuando la tarea finaliza, incluso si no hay salida en pantalla.
 - * -q: Especifica una letra de cola diferente (de a a z) para agrupar los trabajos.
- **head**
 - Se utiliza para mostrar las primeras líneas de un archivo de texto.
 - /usr/bin/head

- Parámetros
 - * -n, -lines=[-]NUM: Muestra las primeras NUM líneas en lugar de las 10 predeterminadas. Si se usa con un signo menos (-n -NUM), imprime todo el archivo excepto las últimas NUM líneas.
 - * -c, -bytes=[-]NUM: Muestra los primeros NUM bytes del archivo. Si se usa con un signo menos (-c -NUM), imprime todo excepto los últimos NUM bytes.
 - * -q, -quiet, -silent: No muestra los encabezados con el nombre del archivo cuando se consultan varios archivos a la vez.
- tail
 - Sirve para mostrar las últimas líneas o partes de un archivo de texto.
 - /usr/bin/tail
 - Parámetros
 - * -n, -lines=[+]NUM: Muestra las últimas NUM líneas en lugar de las 10 predeterminadas. Si se usa con signo más (+NUM), muestra el contenido a partir de la línea número NUM.
 - * -c, -bytes=[+]NUM: Muestra los últimos NUM bytes del archivo. Con signo más (+NUM), muestra desde el byte indicado.
 - * -f, -follow: Monitorea el archivo en tiempo real; muestra las nuevas líneas que se van agregando al final del archivo de forma continua.
 - * -F: Similar a -f, pero realiza un seguimiento reforzado (“retry”), útil si el archivo se rota, se elimina y se vuelve a crear.
 - * -q, -quiet, -silent: No imprime las cabeceras con los nombres de los archivos cuando se consultan múltiples archivos a la vez.

Punto 12

Inciso A

1. Se empieza a ejecutar el código del BIOS.
2. El BIOS ejecuta el POST.
3. El BIOS lee el sector de arranque (MBR).
4. Se carga el gestor de arranque (MBC).
5. El bootloader carga el kernel y el initrd (initial ram disk).
6. Se monta el initrd como sistema de archivos raíz y se inicializan componentes esenciales (por ejemplo, el scheduler).
7. El Kernel ejecuta el proceso init y se desmonta el initrd.
8. Se lee el /etc/inittab.
9. Se ejecutan los scripts apuntados por el runlevel 1.
10. El final del runlevel 1 le indica que vaya al runlevel por defecto.
11. Se ejecutan los scripts apuntados por el runlevel por defecto.
12. El sistema está listo para ser usado.

Inciso B

- El proceso **init** es el único que se considera que no tiene padre, por lo que no es ejecutado por otro proceso. Su función es cargar todos los subprocesos necesarios para el correcto funcionamiento del Sistema Operativo. También es el encargado de montar los filesystems y de hacer disponible los demás dispositivos.

Inciso C

- El *runlevel* es el nivel de ejecución en el que se encuentra el SO. Cada nivel limita la serie de servicios que se pueden ejecutar:

Inciso D

- 0 → halt (parada o apagado).
- 1 → single-user mode (modo monousuario).
- 2 → multi-user without network support (multiusuario sin soporte de red).
- 3 → multi-user console mode (modo multiusuario en consola).

- 4 → N/A (no se utiliza).
- 5 → X11 (modo multiusuario con entorno gráfico basado en X.org).
- 6 → reboot (reinicio)
- Por defecto, Linux se inicia en runlevel 3 o runlevel 5, dependiendo de si la distribución utiliza una interfaz gráfica o no.
 - Esto se define en el archivo `inittab`. Particularmente, hay una directiva que es `initdefault`. El runlevel definido allí es en el que se va a iniciar el sistema. Por ejemplo: `id:5:initdefault` iniciaría en runlevel 5, con interfaz gráfica.

Inciso E

- El archivo `inittab` sirve para dictarle a `init` qué debe hacer, qué programas debe arrancar y qué debe detener dependiendo del estado (runlevel) en el que se encuentre el sistema.
- La estructura de la información almacenada es: `id:runlevels:acción:proceso`
- Contiene las asociaciones de qué scripts deben realizarse al pasar a cada *runlevel*. El directorio de los enlaces a los scripts es `/etc/rcX.d` (donde X es el número de runlevel entre 0 y 6).

Inciso F

- Para cambiar a runlevel Y, se tiene que ejecutar el comando `init Y`, con permisos de `sudo`. El cambio dura hasta que se reinicia la máquina, ya que no se modifica la directiva `initdefault`.

Inciso G

- Los scripts `rcX.d` contienen enlaces a los scripts que se tienen que realizar al cambiar al runlevel X.
- Los scripts en sí se almacenan en `/etc/init.d/`.
- Los enlaces en `rcX.d` tienen este patrón:
 - `[S|K] orden nombre`
 - S y K determinan si dicho script debe ejecutarse o detenerse respectivamente (`start/kill`).
 - El orden es un número de 2 dígitos que establece la prioridad del *script*. Esto determina el orden de ejecución de los mismos, lo que resuelve dependencias entre *scripts*.
 - El nombre es simplemente el nombre lógico que se le da al *script*.

Punto 13

Inciso A

- `Systemd` es un sistema que centraliza la administración de demonios (servicios) y librerías del sistema. Reemplaza a `systemV`, mejorando el paralelismo de arranque.
- Cambios de terminología comparado a `systemV`:
 - *Runlevel* → *target*
 - No utiliza `/etc/inittab`.

Inciso B

- Una *Unit* en `systemd` es una unidad de trabajo. Pueden estar en estado *active* o *inactive* y tienen distintos tipos:
 - *Service*: controla un servicio particular.
 - *Socket*: encapsula IPC, un *socket* del sistema o *file system* FIFO.
 - *Target*: agrupa *units* o establece puntos de sincronización durante el arranque.
 - *Snapshot*: almacena el estado de un conjunto de unidades para que pueda ser restablecido más tarde.

Inciso C

- El comando `systemctl` se utiliza para administrar y controlar los servicios y el estado del sistema en distribuciones de Linux que usan `systemd`. Reemplaza al comando `init`

Inciso D

- En systemd, un *target* agrupa *units* o establece puntos de sincronización durante el arranque. Son el reemplazo de los *runlevels*

Inciso E

- El comando `ps tree` muestra todos los procesos en ejecución en forma de un diagrama de árbol jerárquico

Punto 14

Inciso A

- La información sobre los usuarios en GNU/Linux se almacena en:
 - `/etc/passwd`
 - `/etc/group`
 - `/etc/shadow`

Inciso B

- La UID o user ID es la identificación única de un usuario en Linux. Por lo tanto, no pueden repetirse entre usuarios.
- La GID funciona como identificación única de un grupo de usuarios.

Inciso C

- El usuario root es el usuario administrador del sistema o super-usuario. Solo existe un usuario root, pero se le pueden otorgar permisos de root a un usuario común, agregándolos al archivo `etc/sudoers`.
- La UID del usuario root es 0.

Inciso D

- Detalle paso por paso:
 0. Para ejecutar estos comandos se necesita permisos de *root*. Para obtenerlos hay que acceder como usuario root (no recomendado) o darle permisos de sudo al usuario actual.

1. Crear usuario *isocso*

```
sudo useradd isocso
```

2. Asignar nuevo *home*

```
mkdir isocso
```

```
sudo usermod -d /home/isocso/ isocso
```

Para revisar que haya funcionado (lista los home de todos los usuarios):

```
cat /etc/passwd | grep '/home' | cut -d: -f1,6
```

3. Crear grupo "informática"

```
sudo groupadd informatica
```

4. Añadir usuario al grupo

```
sudo usermod -aG informatica isocso
```

5. Crear archivo (en `/home/isocso`)

```
touch test.txt
```

6. Dar propiedad del archivo al usuario y a su grupo

```
chown isocso:informatica test.txt
```

7. Borrar usuario

```
sudo userdel isocso
```

8. Borrar grupo

```
sudo groupdel informatica
```

9. Ver lista de usuarios

```
cut -d: -f1 /etc/passwd
```

10. Ver lista de grupos

```
getent group
```

Inciso E

Nombre	Significado en inglés	Funcionalidad	Parámetros
useradd	User Add	Comando de bajo nivel para crear un nuevo usuario. Por defecto, solo crea la entrada en /etc/passwd sin configurar carpeta home, contraseña ni shell (depende de la configuración de la distribución).	-m (crea el directorio home) -s /bin/bash (define la shell) -g grupo (define grupo principal)
adduser	Add User	Interfaz de alto nivel y amigable para useradd (muy común en Debian/Ubuntu). Es interactivo: te pregunta paso a paso la contraseña, nombre real y crea automáticamente el directorio home y los archivos base.	--system (crea un usuario de sistema sin home ni shell interactiva)
usermod	User Modify	Modifica las propiedades de una cuenta de usuario ya existente (cambia su home, su shell, su nombre de login, o sus grupos).	-aG grupo (agrega el usuario a un grupo suplementario sin borrar los anteriores) -d /nueva/ruta (cambia el home) -r (elimina también el directorio personal /home/usuario y su buzón de correo)
userdel	User Delete	Elimina la cuenta de un usuario del sistema (borra su entrada de /etc/passwd y /etc/shadow).	-o -l (simula un login completo, cargando las variables de entorno y el path del nuevo usuario)
su	Substitute User / Switch User	Permite cambiar la identidad del usuario actual en la terminal por la de otro usuario (requiere conocer la contraseña del usuario de destino). Si se usa sin argumentos, asume que quieres ser root .	
groupadd	Group Add	Crea un nuevo grupo lógico en el sistema (agrega una entrada en el archivo /etc/group).	-g GID (fuerza la asignación de un ID de grupo numérico específico en lugar de uno automático)

Nombre	Significado en inglés	Funcionalidad	Parámetros
who	Who is logged in	Muestra una lista de los usuarios que están actualmente conectados al sistema, indicando desde qué terminal (tty/pts) y la hora de conexión.	-a (muestra toda la información) -b (muestra la hora del último reinicio)
groupdel	Group Delete	Elimina un grupo existente del sistema. (Nota: No puedes eliminar el grupo principal de un usuario si ese usuario todavía existe).	-
passwd	Password	Permite cambiar la contraseña de un usuario (actualiza el hash en <code>/etc/shadow</code>). Un usuario normal solo puede cambiar la suya propia; el root puede cambiar la de cualquiera.	-l (bloquea la cuenta) -u (desbloquea la cuenta) -e (fuerza al usuario a cambiar su contraseña en el próximo login)

Punto 15

Inciso A

- Los permisos sobre un archivo se dividen entre permisos de lectura, escritura y ejecución. Dichos permisos se definen para el usuario dueño, para el grupo dueño, y para el resto del mundo.

Inciso B

Nombre	Significado en inglés	Funcionalidad	Parámetros (Ejemplos comunes)
chmod	Change Mode	Modifica los permisos de acceso (lectura r , escritura w , ejecución x) de un archivo o directorio. Puede usarse mediante notación octal (números) o simbólica (letras).	-R (recursivo: aplica los cambios a todos los archivos y subdirectorios dentro de una carpeta) +x (notación simbólica: añade permiso de ejecución) 755 (notación octal: rwx para el dueño, rx para grupo y otros)
chown	Change Owner	Cambia el usuario propietario (dueño) de un archivo o directorio. También permite cambiar simultáneamente el grupo propietario.	-R (recursivo: aplica el cambio de dueño a todo el contenido de una carpeta) usuario:grupo (cambia el dueño y el grupo al mismo tiempo, ej: chown isocso:informatica archivo.txt)
chgrp	Change Group	Cambia exclusivamente el grupo propietario de un archivo o directorio.	-R (recursivo: cambia el grupo de todo el contenido de una carpeta)

Inciso C

- La notación octal hace referencia a los permisos UGO-rwx (User, Group, Others y Read, Write, eXecute) del archivo. Esto se ve representado de la siguiente manera mediante un número octal:

- User - Group - Others
- 1 0 1 - 0 0 1 - 0 0 0
- r w x - r w x - r w x
- En el ejemplo anterior, la notación pasada a octal sería 510 (se pasa cada secuencia de 3 bits de binario a octal: 101 -> 5, 001 -> 1, 000 -> 0).
- En este caso, el usuario dueño puede leer y ejecutar el archivo, pero no escribirlo. El grupo dueño solo puede ejecutarlo, y el resto de usuarios o grupos no tienen permiso para nada.

Inciso D

- Sí. El usuario *root* ignora totalmente los permisos de un archivo, y siempre puede ejecutar, leer y escribirlos.

Inciso E

- Un path absoluto es una ruta completa hacia un archivo o directorio, que comienza desde el directorio base (/).
- En cambio, un path relativo es aquel que comienza “desde donde estoy parado”. Desde ahí, si accedo a la carpeta “..” puedo retroceder al directorio anterior.

Inciso F

- El comando para determinar en qué directorio se encuentra actualmente el usuario es **pwd** (*print working directory*).
- Para acceder a tu directorio personal, se puede usar el comando **cd ~**.

Inciso G

Nombre	Significado en inglés	Funcionalidad	Parámetros
mount	Mount	“Montar” un sistema de archivos. Le indica al Kernel que tome un dispositivo de bloque físico (como /dev/sdb1 o un pendrive) y lo conecte lógicamente al árbol del sistema de archivos en una carpeta específica (punto de montaje). Si se ejecuta sin parámetros, lista todo lo que está montado actualmente.	-t ext4 (especifica el tipo de sistema de archivos) -o ro (lo monta en modo Read-Only, solo lectura)
umount	Unmount	“Desmontar”. Desconecta de forma segura un sistema de archivos del árbol de directorios, asegurando que todos los datos en caché (páginas sucias) se vuelquen físicamente al disco antes de cortar el acceso.	-f (fuerza el desmontaje, útil si un disco de red se desconectó) -l (<i>lazy</i> : desconecta lógicamente ahora, limpia después)
du	Disk Usage	Mide el espacio en disco que está consumiendo un archivo o directorio específico. Es ideal para buscar qué carpetas (por ejemplo, dentro de tu home) te están llenando el disco duro.	-s (summary: muestra solo el total de la carpeta, sin detallar cada archivo dentro) -h (human-readable: muestra el tamaño en KB, MB o GB)

Nombre	Significado en inglés	Funcionalidad	Parámetros
df	Disk Free	Muestra la cantidad de espacio libre y usado de todos los sistemas de archivos (particiones) que están montados en el sistema en ese momento, en lugar de carpetas individuales.	-h (human-readable: Muestra megas/gigas en lugar de bloques de 1K) -i (muestra el uso de inodos en lugar de bloques de datos)
fdisk	Format Disk / Fixed Disk	Es una herramienta interactiva para manipular la tabla de particiones de un disco físico (crear, borrar o cambiar el tipo de particiones). Históricamente se usa para el estándar MBR (para GPT suele usarse gdisk o parted).	-l (lista todas las tablas de particiones de todos los discos conectados a la PC sin modificarlas)
mkfs	Make File System	Formatea una partición. Toma una partición física en blanco creada por fdisk (ej. /dev/sdb1) y le inyecta la estructura lógica de un sistema de archivos (bloques, superbloque, tabla de inodos) para que el SO pueda guardar archivos en ella.	-t ext4 /dev/sdb1 (crea un sistema ext4 en sdb1). <i>Suele usarse con sus variantes directas como mkfs.ext4 o mkfs.vfat.</i>
write	Write	(Nota: Este comando es de comunicación, no de discos). Permite enviar un mensaje de texto en tiempo real desde tu terminal hacia la terminal de otro usuario que esté logueado en el mismo sistema.	write usuario_destino (abre un canal interactivo para escribir el mensaje). Termina con Ctrl+D .
losetup	Loop Setup	Asocia un archivo de imagen normal (ej. un .iso o un .img lleno de ceros) a un dispositivo “loop” lógico (ej. /dev/loop0). Esto permite que el kernel trate a ese archivo ordinario como si fuera un disco duro físico real para poder formatearlo o montarlo.	-f (encuentra automáticamente el primer dispositivo loop libre y lo asocia al archivo indicado)
stat	Status	Muestra el estado detallado y completo de un inodo. Imprime metadatos precisos sobre un archivo: su tamaño exacto en bytes, su número de inodo, sus permisos (en octal y texto), y las tres marcas de tiempo (Access, Modify, Change).	-c %a (muestra únicamente los permisos en formato octal, útil para scripts)

Punto 16

Inciso A

- Un proceso en *foreground*, significa que la *shell* no está aceptando nuevos comandos (se bloquea) mientras este se ejecuta. En cambio, un proceso en *background* puede continuar su ejecución mientras se envía un nuevo comando para ejecutar. Por ejemplo, si se ejecuta el comando *sleep* 10, durante 10 segundos se ejecutará en *foreground* el proceso *sleep*, y por eso no se aceptan nuevos comandos hasta que termine.

Inciso B

- Para ejecutar un proceso en *background*, se le agrega el símbolo “&” al final. Por ejemplo: `sleep 10 &`.
- Para traer un proceso del *background* al *foreground*, se ejecuta el comando `fg`. Esto trae el último proceso que se mandó al *background* y lo vuelve a poner en tu pantalla. Ejemplo:
 - `sleep 20 &`
 - `fg`
- Para llevar un proceso del *foreground* al *background*, se hace lo siguiente:
 - Ejecutar un comando en el *foreground* (ej: `sleep 20`)
 - Detenerlo, apretando `Ctrl + Z` en la terminal
 - Ejecutar el comando `bg`

Inciso C

- La finalidad del *pipe* (`|`) es comunicar procesos, conectando de forma directa la salida estándar (*stdout*) del comando de la izquierda con la entrada estándar (*stdin*) del comando de la derecha. Es decir, la salida del comando de la izquierda se convierte en la entrada del de la derecha. Por ejemplo:
 - `ls -l /etc | grep "systemd"`.
 - * El primer comando tiene una salida que es una lista de directorios. En vez de que dicha salida vaya al monitor y se imprima en la terminal, el *pipe* hace que se convierta en la entrada del segundo comando.
 - * Como `grep "systemd"` sirve para filtrar texto, va a recibir la lista de directorios, dejando solo aquellos que contienen la palabra “systemd”. Luego, como no hay más *pipes*, la salida de este último comando se imprime en pantalla.

Inciso D

- Cada proceso tiene 3 *stream* de texto plano. Uno de entrada (*standard input* o *stdin*) y 2 de salida: *standard output* (*stdout*) y *standard error* (*stderr*). Cada uno de estos *streams* tiene una fuente o un destino por defecto:
 - *stdin* es la fuente de entrada para un comando, y por defecto, la recibe del teclado.
 - *stdout* y *stderr* son fuentes de salida de un comando. La primera devuelve la salida esperada por el comando, y la segunda devuelve los mensajes de error, si es que hubo. Ambas, por defecto, se imprimen en el monitor.
 - Cambiar la fuente por defecto de cualquiera de estos *streams*; es decir, hacer que *stdin* reciba el comando por algo que no sea el teclado, o hacer que *stdout* o *stderr* lleven su salida a algo que no sea el monitor, se llama *redirección*. El *pipe* es un ejemplo de redirección, ya que hace que la *stdout* de un proceso vaya a la entrada de otro, en vez de al monitor.
- Estos 3 *streams* de datos tienen un número (*file descriptor*) asociado:
 - *stdin* tiene el FD 0
 - *stdout* tiene el FD 1
 - *stderr* tiene el FD 2
- Hay distintos tipos de redirecciones:
- Redirección de *stdout* (FD 1)
 - `>` redirecciona la salida estándar para llevarlo a un archivo en vez de imprimirlo en pantalla. Si el archivo no existe, lo crea. Si existe, borra todo su contenido y sobrescribe lo deseado.
 - * **Ejemplo:** `ls -l > salida.txt`.
 - `>>` funciona igual que el anterior, pero concatena en vez de sobrescribir. Es decir, si el archivo no existe, lo crea. Si ya existe, agrega la nueva salida al final del archivo sin tocar el contenido original
 - * **Ejemplo:** `ls -l >> salida.txt`

- Redirección de *stderr* (FD 2)
 - Se pueden usar los operandos `>` y `»`, precedidos por un 2.
 - * **Ejemplo:** `ls -l 2> errores.txt` o `ls -l 2>> listado.txt`
- Redirección de *stdout* y *stderr* combinadas
 - Para guardar tanto la salida estándar como la de errores en un archivo, se puede modificar el comando de las siguientes maneras equivalentes entre sí:
 - * `ls -l > salida_y_errores.txt 2>&1`
 - * `ls -l &> salida_y_errores.txt`
- Si cualquier salida, ya sea *stdout* o *stderr*, se envía al dispositivo especial `/dev/null`, el kernel destruye la salida inmediatamente. Sirve para silenciar comandos que tienen salidas ruidosas
 - **Ejemplo:** `comando_ruidoso 2> /dev/null` (El programa corre normalmente, pero todos los mensajes de error son destruidos en silencio para no ensuciar la pantalla)
- Redirección de *stdin* (FD 0)
 - `<` Hace que la entrada sea el contenido directo de un archivo.
 - * **Ejemplo:** `wc -l < listado.txt`
 - * Sin el operando `<`, el programa `wc` recibe el nombre `listado.txt`, lo abre y se ejecuta
 - * Con el operando `<`, la *shell* abre el archivo `listado.txt`, toma todo el texto de ese archivo, y se lo manda a `wc` en forma de *stdin*. El programa `wc` nunca se enterará el nombre del archivo

Punto 17

Inciso A

- El concepto de empaquetar archivos en GNU/Linux es el proceso de tomar múltiples archivos y directorios, y pasarlos a un solo archivo con extensión `.tar`. Esto no reduce el tamaño del archivo en lo absoluto, simplemente agrupa distintos archivos en uno solo, llamado *tarball*

Inciso B

- El tamaño es exactamente igual, justamente porque empaquetar archivos no reduce su tamaño total.

Inciso C

- Para agrupar 4 archivos en uno solo se ejecuta este comando:

```
tar -cf paquete.tar archivo1.txt archivo2.txt archivo3.txt archivo4.txt
```

- Luego, para realmente comprimir ese archivo `paquete.tar`, y que ocupe menos espacio que los originales, se debe comprimir con el siguiente comando:

```
gzip paquete.tar
```

Inciso D

- Sí, se pueden hacer los 2 pasos del Inciso C simultáneamente con este comando:

```
tar -czf archivo_final.tar.gz archivo1.txt archivo2.txt archivo3.txt archivo4.txt
```

Inciso E

Nombre	Significado en inglés	Funcionalidad	Parámetros
tar	Tape Archive	Agrupar (empaquetar) múltiples archivos y directorios en un solo archivo unificado (conocido como <i>tarball</i>). Originalmente diseñado para respaldos en cintas. Por sí solo no reduce el peso, solo agrupa.	<code>-c</code> (crea un empaquetado), <code>-x</code> (extrae), <code>-f</code> (indica el nombre del archivo), <code>-z</code> (comprime con gzip en el mismo paso, ej. <code>tar -czf</code>).

Nombre	Significado en inglés	Funcionalidad	Parámetros
grep	Global Regular Expression Print	Filtra texto. Busca un patrón de caracteres o expresión regular dentro de uno o varios archivos, e imprime en la terminal únicamente las líneas que contienen coincidencias.	-i (ignora mayúsculas y minúsculas), -v (invierte la búsqueda, muestra las líneas que NO coinciden), -r (busca recursivamente dentro de carpetas).
gzip	GNU zip	Comprime archivos individuales mediante el algoritmo DEFLATE para ahorrar espacio en disco. Por defecto, reemplaza el archivo original por uno nuevo con la extensión .gz .	-d (descomprime, logrando lo mismo que el comando gunzip), -k (keep: conserva el archivo original intacto sin borrarlo), -9 (máximo nivel de compresión).
zgrep	Compressed grep	Permite ejecutar el motor de búsqueda de grep directamente en el interior de archivos comprimidos (.gz). Lo hace en la memoria RAM, evitando que tengas que descomprimir el archivo en el disco.	Acepta exactamente los mismos parámetros que grep (ej. zgrep -i "error" log.gz).
wc	Word Count	Analiza un archivo de texto (o el flujo de datos de otro comando) y cuenta la cantidad exacta de saltos de línea, palabras y bytes/caracteres que contiene.	-l (imprime exclusivamente la cantidad de líneas), -w (cuenta solo palabras), -c (cuenta la cantidad de bytes).

Punto 18

- **ls -l > prueba**
 - Escribe la lista de archivos y directorios ubicados en el directorio actual (con detalles) en un archivo “prueba”. Si el archivo no existe, lo crea. Si existe, sobrescribe su contenido
- **ps > PRUEBA**
 - Escribe destructivamente la lista de procesos en ejecución en un archivo “PRUEBA”.
 - Como *Linux* es *case-sensitive*, el archivo “PRUEBA” es totalmente diferente a “prueba”
- **chmod 710 prueba**
 - Modifica los permisos del archivo “prueba” de manera que:
 - * El usuario dueño tiene todos los permisos (lectura, escritura y ejecución) habilitados.
 - * Los usuarios del grupo dueño solo tienen permiso de ejecución
 - * Los usuarios que no son dueños, ni parte del grupo dueño, no pueden leer, escribir, ni ejecutar el archivo.
 - Se puede ejecutar sin *root* si el usuario actual es el dueño del archivo.
- **chown root:root PRUEBA**
 - Este comando requiere permisos de *root*, por lo que no podría ejecutarse en el contexto de la consigna.
 - Lo que está intentando hacer es cambiar el usuario y el grupo dueño del archivo “PRUEBA”, de tal manera que el usuario dueño sea el usuario *root*, y el grupo dueño sea el grupo *root*.
- **chmod 777 PRUEBA**
 - Modifica los permisos del archivo “PRUEBA” de manera que:
 - * Cualquier usuario de cualquier grupo puede leer, escribir y ejecutar el archivo.
- **chmod 700 /etc/passwd**
 - Este comando requiere permisos de *root*, por lo que no podría ejecutarse en el contexto de la consigna.

Para poder ejecutar `chmod`, se debe estar logueado con el usuario dueño del archivo a modificar, pero el archivo `/etc/passwd` es propiedad del sistema.

- Lo que haría es modificar los permisos del archivo, de manera que:
 - * El usuario dueño tiene todos los permisos (lectura, escritura y ejecución) habilitados.
 - * El resto de usuarios no tienen ningún permiso habilitado.
- `passwd root`
 - Como cada usuario solo puede cambiar su propia contraseña, por lo que este comando requiere permisos de *root*
 - Lo que está intentando hacer es iniciar el proceso interactivo para cambiar la contraseña del usuario *root*.
- `rm PRUEBA`
 - Elimina el archivo llamado “PRUEBA”
- `man /etc/shadow`
 - Este comando requiere permisos de *root*, porque el archivo `/etc/shadow` tiene restricciones en su acceso, y es propiedad del usuario *root*.
 - Lo que haría es mostrar en pantalla el contenido del archivo `/etc/shadow`, que contiene los hashes de las contraseñas de todos los usuarios.
- `find / -name * .conf`
 - Su sintaxis es incorrecta. Debería ser: `find / -name "*.conf"`
 - Busca desde el directorio base, aquellos archivos cuyo nombre termina en `.conf`.
- `usermod root -d /home/newroot -L`
 - Este comando requiere permisos de *root*.
 - Está intentando cambiar el home del usuario *root*, para que ahora sea `/home/newroot`. Además, el parámetro `-L` hace que la contraseña del usuario *root* ahora sea irreconocible, por lo que sería imposible entrar de nuevo siendo el usuario *root* (aunque todavía se puede entrar como otro usuario que tenga permisos de `sudo`).
- `cd /root`
 - Este comando requiere permisos de *root*.
 - Está intentando posicionarse dentro de la carpeta `/root`
- `rm *`
 - Elimina todos los archivos del directorio actual (sin incluir los subdirectorios).
- `cd /etc`
 - Se posiciona dentro de la carpeta `/etc`.
- `cp * /home -R`
 - Este comando requiere permisos de *root*, porque la carpeta destino `/home` tiene permisos 755 (111-101-101). Y para copiar algo, se necesitan permisos de escritura en el destino.
 - Lo que haría es copiar todos los archivos y directorios (con subdirectorios incluidos, gracias al `-R`) de la ubicación actual, y pegarlos en `/home`.
- `shutdown`
 - Este comando requiere permisos de *root*.
 - Lo que haría es programar que el sistema se apague en 1 minuto de manera segura.

Punto 19

Inciso A

- `kill 23` o `kill -15 23`
- Kill envía una señal al proceso 23. El `-15` aclara que la señal es para la terminación. Como 15 es el valor por defecto, puede obviarse.

Inciso B

- El proceso *init* o *systemd* tienen PID 1. Sin embargo, por cuestiones de seguridad, *Linux* no permite ejecutar el comando `kill 1`.

Inciso C

- `find /home -name "*.conf*"`
- Asumiendo que todos los archivos de usuario están dentro de `/home`.

Inciso D

- `ps > /home/francisco/procesos`

Inciso E

- `chmod /home/francisco/xxxx 751`

Inciso F

- `chmod /home/francisco/yyyy 650`

Inciso G

- `rm /tmp/*`

Inciso H

- `sudo chown isocso /opt/isodata`

Inciso I

- `pwd >> /home/francisco/donde`

Punto 20

Inciso A

- `su -`

Inciso B

- `useradd -m fbarberena`
- `passwd fbarberena`

Inciso C

- Se modificaron los siguientes archivos:
 - `/etc/passwd`
 - * Se agrega una nueva línea al final con: nombre de login, UID, GID, ruta del *home* y shell predeterminada.
 - `/etc/shadow`
 - * Se agrega una nueva línea con el hash criptográfico de la contraseña asignada.
 - `/etc/group`:
 - * Se añade una nueva línea creando un grupo privado para el usuario (con el mismo nombre, **fbarberena**) y se le asigna un número de GID.
 - `/etc/gshadow`:
 - * Se modifica para gestionar las contraseñas y los administradores del nuevo grupo privado creado en el paso anterior.
- Gracias al parámetro `-m` en *useradd*, se creó un directorio `/home/fbarberena`, que funciona como el nuevo *home* del usuario

Inciso D

- `mkdir /tmp/miCursada`

Inciso E

- `cp -r /var/log/* /tmp/miCursada`

Inciso F

- `chown -R fbarberena:users /tmp/miCursada`

Inciso G

- `chmod -R 723 /tmp/miCursada`

Inciso H

- (En una ventana nueva de la terminal)
- `su - fbarberena`

Inciso I

- `echo $0`

Inciso J

- `ps aux | wc -l`

Inciso K

- `who | wc -l`
 - `who` devuelve una lista de las sesiones activas en el sistema. Luego, `wc -l` toma eso como entrada y cuenta las líneas, que equivale a la cantidad de sesiones.

Inciso L

- `write fbarberena`
 - Ingresar: Voy a apagar el servidor
 - Presionar Ctrl + D.

Inciso M

- `shutdown`

Punto 21

Inciso N

- `mkdir "0273445"`
- **Nota:** no se pueden crear carpetas con “/”.

Inciso O

- `vim LEAME`
- Entrar a modo *insert* apretando la tecla I.
- Escribir nombre, apellido, número de alumno y dirección de correo electrónico, como si fuese un .txt normal (se puede usar el Enter para saltar de línea)
- Apretar Esc para entrar a modo normal. Guardar el archivo con el siguiente comando:
- `:wq`

Inciso P

- `chmod 017 /home/francisco/LEAME`

Inciso Q

- `ls > /home/francisco/leame`
- Es posible porque *Linux* es *case-sensitive*, “leame” es un archivo totalmente distinto a “LEAME”

Inciso R

- Para localizar un archivo dentro del *file system*, usaría el comando *locate*, ya que utiliza una base de datos interna indexada con todos los nombres de los archivos, lo que es más rápido que buscar directorio por directorio.
- Sin embargo, *locate* solo permite buscar por nombre. Si quisiera localizar varios archivos con características similares, usaría el comando *find*, que es más lento pero permite buscar por otros criterios además del nombre (tamaño, fecha de modificación, permisos, o usuario dueño).

Inciso S

- `locate "*.so" > /home/francisco/0273445/ejercicioF`
- **Nota:** si prueban esto con `find / -name "*.so"`, se van a dar cuenta que la diferencia en velocidad mencionada en el inciso R es significativa.

Punto 22

- Estas corresponden a las anotaciones del archivo `explicacion_de_ejercicio`, creado originalmente en Vim.
 - **Nota:** hay comandos con más de una parte, por eso algunos tienen un número entre paréntesis antes de la explicación.
1. Crea una carpeta llamada `iso`.
 2. (1) Entra a la carpeta `iso`.
 3. (2) Ejecuta el comando `ps`, y escribe su salida en un archivo llamado `f0`, sobrescribiendo su contenido.
 4. Lista todo el contenido dentro del directorio “iso”, y vuelca ese listado en un archivo llamado `f1`, sobrescribiendo su contenido.
 5. Se posiciona en el directorio raíz.
 6. Imprime el directorio asignado como personal del usuario actual (ej: `/home/francisco`).
 7. (*Asumo que el comando pretendía ser `ls -l > $HOME/iso/ls`*) Lista detalladamente todos los archivos del directorio actual (`/`). Luego, vuelca ese contenido en un archivo llamado `ls` de manera destructiva. Dicho archivo se crea/encuentra dentro de la carpeta `iso` creada previamente en el *home* del usuario.
 8. (1) Se posiciona en el directorio *home* del usuario.
 9. (2) Crea una carpeta llamada `f2`.
 10. Lista de manera detallada los permisos, dueño, etc de la carpeta `f2` (no de sus archivos o subdirectorios). Esto es debido al parámetro `-d` (*directory*).
 11. Cambia los permisos de `f2`, de manera que: el usuario dueño puede escribir y ejecutar, el grupo dueño puede leer su contenido, y cualquier otro usuario solo puede ejecutarlo.
 12. Crea un archivo vacío llamado `dir`.
 13. Entra a la carpeta `f2`.
 14. Vuelve al *home*, y entra a la carpeta `iso`.
 15. Imprime el directorio actual (`/home/francisco/iso`) en un nuevo archivo llamado `f3`, de forma destructiva.
 16. (1) Imprime todos los procesos en ejecución en la terminal actual.
 17. (2) Recibe la salida de 14.1, y devuelve solamente aquellas líneas que contienen la palabra `ps`.
 18. (3) Recibe la salida de 14.2, y devuelve cuántas líneas tiene (1).
 19. (4) Vuelca la salida de 14.3 en un archivo llamado `f3` de manera no destructiva. Dicho archivo se encuentra en `/home/francisco/f2`. Esto es posible sin acceso a *root*, porque previamente definimos que el usuario dueño tiene permisos de escritura sobre el directorio `f2`.

20. (1) Actualiza los permisos del directorio f2, para que el usuario dueño tenga acceso total, y el resto de usuarios no tengan ningún permiso.
21. (2) Retrocede al *home* del usuario.
22. Busca, desde el *home* del usuario, un archivo cuyo nombre sea **etc/passwd**. Sin embargo, ningún archivo puede tener ese nombre. Para buscar por nombre completo (con ubicación incluida), se debe usar el parámetro **-wholename**.
23. Lo mismo que en el 16 pero iniciando desde el directorio raíz.
24. Crea una carpeta llamada ejercicio5.
25. Asumo que “el directorio creado en 24a” se refiere a la carpeta creada en el inciso 18, llamada “ejercicio5”. El comando para hacer eso sería: `cp -r /home/francisco/iso /home/francisco/ejercicio5`.
26. Los archivos restantes son: dir, ejercicioF, leame, LEAME y el directorio f2. Para copiar los archivos y directorios se usa:
 - `cp dir ejercicioF leame LEAME f2 /home/francisco/ejercicio5`.

Punto 23

- Para crear el esquema de directorios, ejecute en la terminal el archivo crearEsquema23.sh.

Inciso A

- `mv dir1/f3 ~`

Inciso B

- `cp dir2/f4 dir1/dir11`

Inciso C

- `cp dir2/f4 dir1/dir11/f7`

Inciso D

- `mkdir copia; cp -r dir1/* copia`

Inciso E

- `mv f1 archivo; ls -l archivo`

Inciso F

- `chmod 617 archivo`

Inciso G

- `rename 's/$/.exe/' f3 dir2/f4`
- Lo que hace el comando *rename* es:
 - `s/buscar_esto/reemplazar_con_esto/` archivo1 archivo2 archivo3.
 - La *s* significa sustitución.
 - El *\$* significa “El punto final del archivo”

Inciso H

- `chmod 023 f3.exe dir2/f4.exe`

Punto 24

Inciso A

- `mkdir /tmp/logs`

Inciso B

- `cp -r /var/log/* /tmp/logs`

Inciso C

- `tar -cf misLogs.tar /tmp/logs`

Inciso D

- `tar -czf misLogs.tar.gz /tmp/logs`

Inciso E

- *En realidad ya estarían almacenados en el directorio personal del usuario*
- `cp misLogs.tar misLogs.tar.gz ~`

Inciso F

- `rm -r /tmp/logs`

Inciso G

- `mkdir desempaquet1; tar -xf misLogs.tar -C ~/desempaquet1`
- `mkdir desempaquete2; tar -xzf misLogs.tar.gz -C ~/desempaquete2`
- La flag `-C` sirve para cambiar el directorio en donde se extrae.